DRONE SOIL-SAMPLING ROUTE OPTIMIZATION

Project Documentation

A C++ implementation of the Travelling Salesperson Problem using Nearest Neighbor and Branch and Bound

Prepared from the supplied source code
Language: C++ | Visualization: Graphviz
Date: November 2026

Document Control

Project	Drone Soil-Sampling Route Optimization
Document	Technical documentation and user guide
Implementation	C++ (standard library)
Primary methods	Nearest Neighbor heuristic; Branch and Bound exact search

1. Project Overview

This project finds a low-cost closed route that starts at a designated base location, visits every sampling location exactly once, and returns to the base. It models the task as a Travelling Salesperson Problem (TSP). The program accepts either two-dimensional coordinates or a complete distance/cost matrix, calculates a Nearest Neighbor baseline, then applies Branch and Bound to search for an optimal tour under the supplied costs.

The program also reports search statistics and execution times, optionally verifies the result with brute force when there are exactly eight locations, and writes a Graphviz DOT file to visualize the selected route.

2. Problem Definition

Given n locations and a base vertex b, find a permutation of all locations such that:

- the route begins at b;
- each location is visited exactly once;
- the route ends at b; and
- the sum of the directed travel costs along the route is minimized.

For a route $r = (b, v_1, \ldots, v_{n-1}, b)$, its total cost is:

$$\mathbf{Cost(r) = d[b][v_1] + d[v_1][v_2] + \ldots + d[v_{n-1}][b]}$$

If d[i][j] = d[j][i] for all pairs, the instance is symmetric. Otherwise it is asymmetric (directed). The implementation uses the matrix as supplied and therefore supports both types, assuming all relevant travel costs are finite and non-negative.

3. Objectives

- Generate a feasible route quickly using the Nearest Neighbor heuristic.
- Improve or certify the route cost using Branch and Bound.
- Compare heuristic and exact-search costs and execution times.
- Track explored nodes and pruned branches to illustrate search behavior.
- Visualize the final tour using Graphviz.

4. Software and Hardware Requirements

Item	Requirement
Language/compiler	C++11 or later; a compiler such as g++, Clang, or MSVC
Standard libraries	bits/stdc++.h is supported by GCC; use individual standard headers for portability
Visualization	Graphviz installed and the neato command available

Input size	Program accepts 1 to 20 locations; practical exact-search size depends strongly on the instance
Operating system	Any platform with a compatible C++ compiler; Graphviz is optional

5. System Workflow

1. Select input mode: coordinates or distance matrix.
2. Enter the number of locations and the input data.
3. For coordinate input, calculate pairwise Euclidean distances.
4. Enter and validate the base location index.
5. Display the distance matrix and classify it as symmetric or asymmetric.
6. Run Nearest Neighbor and record its route, cost, and elapsed time.
7. Initialize Branch and Bound with the Nearest Neighbor tour as the incumbent solution.
8. Explore candidate routes recursively, pruning branches whose lower bound cannot beat the incumbent.
9. Display the best route, cost, explored nodes, pruned branches, and execution time.
10. For n = 8, run brute-force verification; finally write the DOT visualization.

6. Module Description

Function / module	Responsibility
Point structure	Stores an x and y coordinate for a sampling location.
TSP constructor	Stores the distance matrix, derives n, and records the chosen base vertex.
nearestNeighbor()	Builds a greedy feasible tour by repeatedly choosing the nearest unvisited location, then returns to base.
lowerBound()	Computes an optimistic completion-cost estimate from the current partial route and remaining vertices.
dfs()	Performs recursive Branch and Bound search, exploring nearby candidates first and pruning non-promising branches.
branchAndBound()	Resets counters, seeds the incumbent with Nearest Neighbor, creates the initial unvisited set, and starts DFS.
bruteForce()	Enumerates every permutation of non-base vertices and returns the minimum-cost tour.
isSymmetric()	Checks whether the distance matrix is symmetric within EPS tolerance.
generateVisualization()	Writes a directed Graphviz DOT graph and prints the command to render it.
main()	Handles user input, validation, algorithm execution, reporting, verification, and visualization.

7. Algorithms

7.1 Nearest Neighbor heuristic

Start at the base. At each step, select the unvisited vertex with the smallest outgoing cost from the current vertex. After all vertices have been visited, add the return edge to the base. This produces a valid tour quickly, but it is not guaranteed to be optimal.

Time complexity: $O(n^2)$ for the implementation using a linear scan at each step.

7.2 Branch and Bound

Branch and Bound explores partial tours. The current best complete tour is the incumbent upper bound. For each partial route, lowerBound() estimates a minimum possible completion cost. If that estimate is greater than or equal to the incumbent (within EPS), the branch is discarded because it cannot improve the known solution.

- Initialization: use the Nearest Neighbor route as the first complete solution.
- Branching: choose each remaining vertex as the next stop.
- Candidate ordering: examine vertices in increasing distance from the current vertex, often finding good tours earlier.
- Bounding: add a minimum required outgoing edge estimate for the current vertex and every unvisited vertex.
- Termination: when no vertices remain, close the tour to the base and update the incumbent if it is better.

The bound is a relaxation: it independently chooses a cheapest permitted outgoing edge for each source among remaining vertices or the base. Because it can allow choices that cannot all coexist in one Hamiltonian tour, it is optimistic and is intended not to exceed the cost of any feasible completion. This supports safe pruning for finite, non-negative complete cost matrices.

7.3 Brute Force verification

Brute force fixes the base and enumerates all $(n-1)!$ permutations of the other vertices. It computes each closed-tour cost and keeps the minimum. It is used only when n = 8 in main(), providing an independent check of the Branch and Bound result.

8. Input and Output Specification

8.1 Coordinate mode

Enter n coordinate pairs (x, y). The program calculates Euclidean distance using $d[i][j] = \text{sqrt}((x_i - x_j)^2 + (y_i - y_j)^2)$. The resulting matrix is symmetric.

8.2 Distance-matrix mode

Enter n rows of n numeric values. Entry d[i][j] is the cost of travelling from i to j. The matrix may be asymmetric. Negative entries are rejected by main().

8.3 Output

- Formatted distance matrix and symmetric/asymmetric classification.
- Nearest Neighbor route, total cost, and execution time in milliseconds.
- Branch and Bound optimal route, best cost, explored-node count, pruned-branch count, and execution time.
- Brute-force route, cost, runtime, and pass/fail comparison when n = 8.
- A file named drone_route.dot and a suggested Graphviz rendering command.

9. Build and Run Instructions

Save the source as drone_tsp.cpp. With GCC, compile using:

```
g++ -std=c++17 -O2 -o drone_tsp drone_tsp.cpp
```

Run on Linux/macOS:

```
./drone_tsp
```

Run on Windows (MinGW):

```
drone_tsp.exe
```

If Graphviz is installed, render the generated route:

```
neato -Tpng drone_route.dot -o drone_route.png
```

10. Example Input

A small coordinate-mode example (five locations, base index 0):

```
1
5
0 0
3 0
4 3
0 4
-2 2
0
```

The program will derive the distance matrix, run both algorithms, and create the DOT file. The exact route depends on the input distances and tie-breaking order.

11. Complexity Analysis

Component	Time complexity	Notes
Distance matrix from coordinates	$O(n^2)$	Every ordered pair is calculated.
Nearest Neighbor	$O(n^2)$	Linear scan for the next unvisited vertex.
One lower-bound evaluation	$O(n^2)$	For each source, scan remaining destinations.
Branch and Bound	$O(n! \cdot n^2)$ worst-case upper estimate	Pruning can reduce practical work, but worst case remains factorial.
Brute force	$O((n-1)! \cdot n)$	Enumerates every tour permutation.
Stored distance matrix	$O(n^2)$	Plus O(n) route and recursion state.

The advertised input limit of 20 is a validation limit, not a guarantee that exact search will finish quickly for every 20-location instance. Branch and Bound remains exponential in the worst case.

12. Correctness Considerations

- Every recursive branch removes the selected next vertex from unvisited before descending, so a vertex cannot be selected twice on one route.
- A complete route is accepted only after all non-base vertices have been visited and the return edge to base is included.
- The incumbent is initialized with a valid Nearest Neighbor tour, so pruning always compares against a feasible upper bound.
- The lower bound is optimistic when the matrix represents a complete directed graph with finite non-negative costs; pruning at bound ≥ incumbent cannot remove a strictly better tour.
- Brute-force comparison at n = 8 provides a useful independent validation case.

13. Testing Plan

Test case	Expected behavior	Purpose
One location	Route is base → base; cost is the diagonal/zero return cost.	Boundary case.
Two locations	Visits the other location and returns to base.	Smallest non-trivial tour.
Symmetric coordinate input	Matrix is reported symmetric.	Euclidean distance generation.
Asymmetric matrix	Matrix is reported asymmetric; directed costs are used.	Directed TSP support.
Negative matrix entry	Program displays an error and exits.	Input validation.
Invalid n or base	Program rejects out-of-range values.	Input validation.
Eight locations	Branch and Bound cost equals brute-force cost within EPS.	Optimality cross-check.
Several equal edge costs	Program still returns a valid minimum-cost tour.	Tie handling.

14. Limitations and Recommended Improvements

- The program assumes a complete matrix of usable finite costs. It does not represent missing routes or forbidden edges explicitly.
- The 20-location input cap may still be too high for an exact factorial search; runtime can grow rapidly.
- The current lower bound is simple and may be weak. Stronger assignment, minimum spanning tree (for symmetric TSP), or reduced-cost bounds can improve pruning.
- The code uses double and EPS comparisons. For integer-valued distances, scaled integers can avoid floating-point comparison concerns.
- The code uses <bits/stdc++.h>, which is convenient with GCC but is not a standard C++ header; portable projects should include required headers individually.
- Input stream failures and non-finite values (NaN or infinity) are not fully validated.
- The visualization uses directed edges. Coordinates are written when coordinate mode is selected; matrix mode relies on Graphviz layout.
- For n = 1, the route is printed as base → base and the cost is zero when the diagonal entry is zero.

15. Future Enhancements

- Add robust validation for malformed, non-finite, and incomplete matrices.
- Allow forbidden links and unreachable vertices with explicit infinity handling.
- Implement stronger lower bounds and memoization/dynamic programming for suitable sizes.
- Add configurable time limits and report whether optimality was proven before timeout.
- Export route and metrics to CSV/JSON and provide richer route labels in Graphviz.
- Integrate real-world drone constraints such as battery capacity, no-fly zones, wind, sampling duration, and recharge stations.
- Add a GUI or map-based input interface.

16. Conclusion

The project demonstrates how a practical heuristic and an exact optimization method can be combined for drone soil-sampling route planning. Nearest Neighbor quickly supplies a valid initial tour, while Branch and Bound searches for a better tour and can prove optimality when it completes. Brute-force verification for eight locations and Graphviz visualization provide additional validation and interpretability. For larger instances, stronger bounds or alternative optimization methods may be required.

Appendix A. Important Implementation Notes

The lowerBound() method adds one minimum outgoing-edge estimate for the current vertex and each unvisited vertex. When no vertices remain, it explicitly uses the current-to-base edge. The DFS checks this bound before processing a leaf; if the bound equals the incumbent, the leaf may be pruned, which is safe because the incumbent already has that cost.

The nearestNeighbor() baseline always appends the base to close the route. The program therefore prints a closed route for both the heuristic and exact-search outputs.